

Project Proposal: Enemy Missile Destruction System Using Graph Algorithm

1. Project Title:

Enemy Missile Destruction System Using Graph Algorithm

2. Objective:

The goal of this project is to create an intelligent missile defense system using **graph-based algorithms**. The system will determine the **shortest and most efficient path** to intercept enemy missiles while considering obstacles such as enemy defenses and resource constraints.

3. Problem Statement:

Modern warfare requires **precise and efficient missile guidance** to neutralize threats. However, enemy defense systems and high-cost paths make interception challenging. This project will use **Dijkstra's Algorithm** to determine the most effective attack strategy, optimizing both time and resources.

4. Scope of the Project:

- Represent military locations as a **graph**, where:
 - **Nodes** represent locations (military bases, enemy bases, etc.).
 - **Edges** represent possible paths with different weights (distances, obstacles, costs).
- Use **Dijkstra's Algorithm** to determine the **shortest and safest** path.
- Identify enemy defense barriers that may **block missile paths**.
- Evaluate if the missile can **successfully reach its target** under the given constraints.

5. Methodology:

Graph Representation:

- **Military bases, enemy bases, and paths** are modeled as a **weighted graph**.
- Each path (edge) has a weight, which represents the **difficulty or cost** of taking that path.
- Enemy defense systems are integrated as **obstacles**.

Algorithm Selection:

- **Dijkstra's Algorithm** is used to find the **shortest path** to intercept the enemy missile.
- A **priority queue (Min-Heap)** is used to optimize performance.

How Dijkstra's Algorithm Works:

Dijkstra's Algorithm is used to find the shortest path from a source node to all other nodes in a graph. It works as follows:

1. **Initialize Distances:** Set the distance of all nodes to **infinity (INF)**, except for the **starting node**, which is set to **0**.
2. **Select the Minimum Distance Node:** Find the unvisited node with the smallest known distance.
3. **Update Neighboring Nodes:** Check all neighbors of the selected node. If moving to a neighbor reduces the known distance, update it.
4. **Mark the Node as Visited:** Once a node has been processed, mark it as visited.
5. **Repeat Until All Nodes are Processed:** Continue the process until all nodes have been visited or no more nodes can be reached.
6. **Backtrack to Find the Shortest Path:** Use the previous node array to reconstruct the shortest path from the source to the destination

Decision Making:

- If a **shortest path is found**, the missile is launched.
- If the path **passes through an enemy defense**, the missile is intercepted, and the mission fails.
- If no path is found or the cost is **too high**, the mission is considered infeasible.

Technologies and Tools Used:

- **Programming Language:** C
- **Graph Algorithms:** Dijkstra's Algorithm
- **Data Structures:** Graph, Priority Queue, and Adjacency List

Code Implementation with Explanation:

Including Header Files

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <limits.h>
```

- `stdio.h` is used for input/output operations.
- `stdlib.h` is used for memory management.

- `limits.h` provides the constant `INT_MAX`, which represents infinity.

Defining Constants

```
#define MAX_NODES
```

```
#define INF INT_MAX
```

- `MAX_NODES` limits the number of locations.
- `INF` represents an unreachable state.

Defining Edge Structure

```
typedef struct {
```

```
    int source;
```

```
    int dest;
```

```
    int weight;
```

```
} Edge;
```

- This structure represents an **edge** in the graph, storing:
 - `source`: Start location.
 - `dest`: Destination location.
 - `weight`: The cost/distance to travel.

Global Variables

```
Edge edges[MAX_NODES * MAX_NODES];
```

```
int edge_count = 0;
```

```
int num_nodes;
```

```
int enemy_defense = -1;
```

- `edges` stores all possible paths between locations.
- `edge_count` keeps track of the number of edges.

- `num_nodes` stores the total number of locations.
- `enemy_defense` stores the position of enemy defenses.

Adding Edges to the Graph

```
void addEdge(int src, int dest, int weight) {
```

```
    edges[edge_count].source = src;
```

```
    edges[edge_count].dest = dest;
```

```
    edges[edge_count].weight = weight;
```

```
    edge_count++;
```

- } This function adds a path to the graph.
- It stores the **source, destination, and weight** in the `edges` array.

Implementing Dijkstra's Algorithm

```
void shortestPath(int start, int target) {
```

```
    int distance[MAX_NODES], previous[MAX_NODES];
```

```
    int visited[MAX_NODES] = {0};
```

- `distance[]` stores the shortest known distance to each node.
- `previous[]` stores the previous node in the path.
- `visited[]` keeps track of nodes already processed.

```
    for (int i = 0; i < num_nodes; i++) {
```

```
        distance[i] = INF;
```

```
        previous[i] = -1;
```

```
    }
```

Initialize all distances to `INF` (unreachable) and set previous nodes to `-1`.

Finding the Minimum Distance Node

```

for (int i = 0; i < num_nodes - 1; i++) {
    int minDist = INF, minIndex = -1;

    for (int j = 0; j < num_nodes; j++) {
        if (!visited[j] && distance[j] < minDist) {
            minDist = distance[j];
            minIndex = j;
        }
    }
}

```

- } This loop selects the **unvisited node** with the **smallest distance**.
- If no such node is found, it **exits the loop**.

Updating Neighboring Nodes

```

visited[minIndex] = 1;

for (int j = 0; j < edge_count; j++) {
    int u = edges[j].source, v = edges[j].dest, w = edges[j].weight;

    if (u == minIndex && !visited[v] && distance[u] != INF && distance[u] +
w < distance[v]) {
        distance[v] = distance[u] + w;
        previous[v] = u;
    }
}
}

```

- Updates the distance of all **neighboring nodes**.

- If a **shorter path** is found, it updates the distance and sets the previous node.

Checking Mission Status

```

if (distance[target] == INF) {
    printf("No possible path found! Mission failed!\n");
} else if (distance[target] > 100) {
    printf("Path found but too costly (%d). Mission failed!\n", distance[target]);
} else {
    printf("Enemy missile destroyed! Shortest path: ");
    for (int at = target; at != -1; at = previous[at]) {
        printf("%d ", at);
    }
    printf("\n");
}
}

```

- **If no path is found**, the mission fails.
- **If the path is too costly**, it is rejected.
- Otherwise, it prints the **shortest path** and confirms enemy missile destruction.

Sample Input and Output

Input:

Enter number of locations (nodes): 5

Enter number of paths (edges): 6

Enter paths (source destination weight):

0 1 10

0 2 5

1 2 2

1 3 1

2 3 9

3 4 4

Enter missile launch base: 0

Enter enemy missile base: 4

Enter enemy defense system location (-1 if none): 2

Output:

== Mission Status ==

Missile intercepted at node 2! Mission failed!

Alternative Output (No Defense in Path):

=== Mission Status ===

Shortest path to destroy enemy missile: 0 -> 2 -> 1 -> 3 -> 4

Enemy missile destroyed!

Complexity Analysis:

Time Complexity:

- **Dijkstra's Algorithm (Without Priority Queue):** $O(V^2 + E)$
- **Dijkstra's Algorithm (With Min-Heap):** $O((V + E) \log V)$
 - V = Number of nodes (military locations)
 - E = Number of edges (paths)

Space Complexity:

- $O(V + E)$ due to adjacency list representation.
- Additional $O(V)$ space for `distance[]` and `visited[]` arrays.

Expected Outcomes:

- The system will **calculate the shortest path** for missile interception.
- It will determine if the missile can **reach its target**.
- The system will provide **real-time mission success or failure status**.

Future Enhancements:

- **Machine Learning Integration** to predict enemy defenses dynamically.
- **Real-time Graph Updates** based on battlefield changes.
- **Multiple Missile Support** for strategic interception.

This proposal provides a detailed explanation of both **theoretical and coding aspects** of the project.